

Informe de Entrega - Cristian Rosero (Perfil Publico de Usuario)

Para: Cristian Rosero **De:** Daniel Palacios (Lider backend) **Modulo asignado:** Ver perfil publico de un usuario (item 58 de la metodologia) **Fecha de revision:** 6 de agosto de 2026 **Requerimiento:** RF104 (REVISION) **Estado:** Entregado e integrado con correcciones

1. Resumen

Cristian, recibí tu trabajo del modulo de **perfil publico de usuario**. Lo revise completo, lo probe y lo integre al backend real del proyecto. Tu modulo funciona y cumple el requerimiento RF104. Aplique algunas correcciones de seguridad y estructura para que cumpla las reglas del proyecto; en esta nota te explico que hiciste bien y que ajuste.

2. Lo que hiciste bien

1. **Endpoint funcional:** implementaste `GET /perfil-publico/:id` consultando la tabla `usuarios` por id.
2. **Consulta parametrizada:** usaste `?` en el SQL, sin concatenar strings. Correcto, eso evita SQL injection.
3. **Exclusion de datos sensibles:** tu SELECT no incluia `password`, `token_recuperacion` ni `direccion_envio`. Muy bien.
4. **Mapeo de campos correcto:** las columnas que usaste (`nombre_completo`, `foto_perfil`, `descripcion_personal`) existen en el esquema oficial `commercy_v2`. Lo verifique contra el schema. Esto evita la brecha critica B1 que tuvimos con el carrito.
5. **Conexion con el frontend:** el componente PerfilVendedor consume tu endpoint con `fetch` y `useParams`. Correcto.

3. Correcciones que aplique a tu entrega

Tu trabajo estaba bien encaminado, pero le hice estos ajustes para cumplir las reglas del proyecto:

Lo que tenias	Lo que ajuste	Por que
Credenciales de BD hardcodeadas en <code>db.js</code> (<code>localhost/root</code>)	Uso del pool real de <code>config/db.js</code> (credenciales desde <code>.env</code>)	Las credenciales nunca van en el codigo; se leen del entorno
En el error 500 devolvias <code>error.message</code> al cliente	Lo dejo en log interno y devuelvo respuesta generica	No se exponen detalles internos del servidor en produccion
Respuesta plana <code>{ mensaje: ... }</code>	Formato estandar <code>{ success, data }</code> y <code>{ success, error: { code, message } }</code>	Todos los endpoints del proyecto usan este contrato
Parametro <code>id</code> sin validar	Valido que sea entero positivo (400 <code>VALIDATION_ERROR</code>)	Evita peticiones invalidas
Exponias el <code>email</code> en el perfil publico	Lo elimine del perfil publico	El email es dato sensible; el perfil publico no debe mostrarlo
Usuarios baneados se veian	Si <code>activo = 0</code> devuelvo 404	Un usuario baneado no tiene perfil publico visible
Traias <code>db.js</code> y <code>server.js</code> propios	No los integre; use la estructura del proyecto (<code>config/db.js</code> , <code>app.js</code>)	Cada integrante entrega solo su modulo, no la infraestructura
Ruta montada en la raiz	La monto en <code>/api/usuarios/perfil- publico/:id</code>	Convencion de rutas del proyecto (prefijo <code>/api</code>)

Archivos que modifique en el backend

Archivo	Cambio
<code>backend/src/server/controllers/usuarios.controllers.js</code>	Tu <code>getPerfilPublico</code> corregido con validacion y seguridad

Archivo	Cambio
<code>backend/src/server/routes/usuarios.routes.js</code>	Router del modulo usuarios
<code>backend/src/server/app.js</code>	Montaje de <code>/api/usuarios</code>
<code>backend/src/server/__tests__/usuarios.controllers.test.js</code>	7 tests unitarios

4. Testing de tu modulo

Cree 7 tests unitarios (Vitest + Supertest, mock de BD sin conexion real):

Caso probado	Resultado
id no entero positivo (<code>abc</code>)	400 VALIDATION_ERROR
usuario inexistente (999)	404 NOT_FOUND
usuario baneado (<code>activo = 0</code>)	404 NOT_FOUND
perfil publico correcto	200 con nombre, biografia, avatar
avatar por defecto sin foto	200 con placeholder
fallo de BD	500 INTERNAL_ERROR sin detalles
consulta parametrizada	<code>WHERE id = ?</code> con parametro <code>[42]</code>

Resultado global

```

Test Files  2 passed (2)
  Tests    26 passed (26)    (19 carrito + 7 tuyos)

Statements : 94.16%   Branches : 87.3%
Functions  : 100%    Lines    : 94.11%
```

La cobertura del proyecto subio de 93% a 94.16% con tu modulo (minimo exigido: 60%).

5. Contrato del endpoint final

```
GET /api/usuarios/perfil-publico/:id
```

Respuesta exitosa (200):

```
{
  "success": true,
  "data": {
    "id": 7,
    "nombre": "Maria Lopez",
    "biografia": "Vendedora de accesorios",
    "avatar": "https://img.com/maria.png"
  }
}
```

Errores:

```
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "El id debe ser un número" } }
{ "success": false, "error": { "code": "NOT_FOUND", "message": "Usuario no encontrado" } }
{ "success": false, "error": { "code": "INTERNAL_ERROR", "message": "Error al obtener el perfil" } }
```

6. Tu entrega quedo ordenada

Tu carpeta original tenia el proyecto completo clonado (frontend con `node_modules`, etc.). La organice dejando solo tu modulo:

```
AVANCES/CRISTIAN ROSERO/
├─ backend/
│   └─ src/
│       └─ server/
│           ├── controllers/usuarios.controllers.js
│           └─ routes/routes.js
└─ Documentacion.txt
```

7. Conclusion

Tu modulo quedo integrado, probado y registrado en el changelog del proyecto, con el commit atribuido a tu nombre. Las correcciones fueron de seguridad y estructura; tu trabajo funcional se mantuvo intacto. Cualquier duda me escribes.